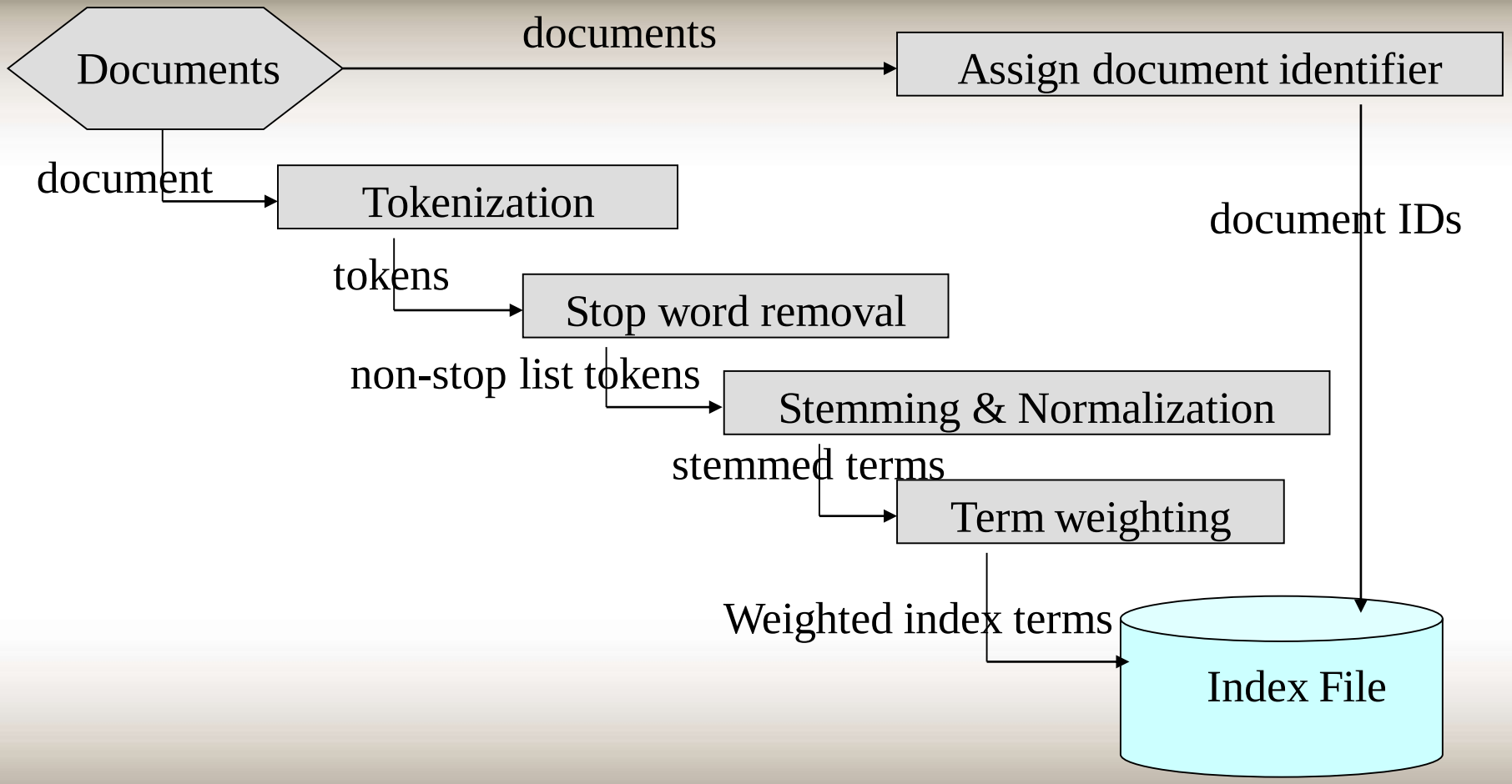
Chapter 3 : Index Construction

Adama Science and Technology University
School of Electrical Engineering and Computing
Department of CSE
Dr. Mesfin Abebe Haile (2024)

fppt.com

Indexing Subsystem



fppt.com

Indexing: Basic Concepts

- Indexing is used to *speed up access to desired information* from *document collection* as per users query such that:
  - It enhances efficiency in terms of time for retrieval. Relevant documents are searched and retrieved quick.
  - Example: author catalog in library.
- An index file consists of records, called *index entries*.

- Index files are *much smaller* than the original file.
  - Remember Heaps Law: In 1 GB text collection the size of a vocabulary is only 5 MB (Baeza-Yates and Ribeiro-Neto, 2005)
  - This *size may be further reduced* by Linguistic pre-processing (like stemming & other normalization methods).
- The *usual unit* for indexing is the *word*.
  - Index terms - are used to look up records in a file.

fppt.com

How Current Search Engines index?

- Indexes are built using a *web crawler*, which *retrieves each page* on the Web for *indexing*.
  - After indexing, the local copy of each page is discarded, unless stored in a cache.
- Some search engines: *automatically index*
  - Such *search engines* include: Google, AltaVista, Excite, HotBot, InfoSeek, Lycos.
- Some others: *semi automatically index*
  - Partially *human indexed*, hierarchically organized.
  - Such *search engines* include: Yahoo, Magellan, Galaxy, WWW Virtual Library.
- Common features
  - allow *Boolean searches*.

fppt.com

Major Steps in Index Construction

- *Source file*: Collection of text document.
  - A document can be described by a set of representative keywords *called index terms*.
- Index Terms Selection:
  - *Tokenize*: identify words in a document, so that each document is represented by a list of keywords or attributes
  - *Stop words*: removal of high frequency words
    - Stop list of words is used for comparing the input text
  - *Word stem and normalization*: reduce words with similar meaning into their stem/root word. *Suffix stripping* is the common method.
  - *Term relevance weight*: Different index terms have varying relevance when used to describe document contents.
    - This effect is captured through the assignment of *numerical weights* to each index term. There are different weighting methods: TF, TF*IDF, …
- *Output*: a set of *index terms* (vocabulary) to be used for *Indexing the documents* that each term occurs in.

fppt.com

Basic Indexing Process

Documents to be indexed.

Friends, Romans, countrymen.

⋮

Token stream.

Tokenizer

Friends Romans countrymen

Modified tokens.

Linguistic preprocessing

friend roman countryman

Index File (Inverted file).

Indexer

friend → 2 → 4 →

roman → 1 → 2 →

countryman → 13 → 16

fppt.com

Building Index file

- An *index file* of a document is a file consisting of *a list of index terms* and *a link to one or more documents* that has the index term.
  - A *good index file* maps each keyword K_i to a set of documents D_i *that contain the keyword*.

- Index file usually has *index terms* in a *sorted order*.
  - The *sort order* of the terms in the index file provides an order on a *physical file*.

fppt.com

Building Index file

- An index file is list of search terms that are organized for *associative look-up*, i.e., to answer user's query:
  - In which documents does a specified *search term appear*?
  - Where within each document *does each term appear*? (There may be several occurrences.)

- For organizing index file for a collection of documents, *there are various options available*:
  - Decide what *data structure* and/or *file structure* to use.
  - Is it sequential file, inverted file, suffix array, signature file, etc. ?

fppt.com

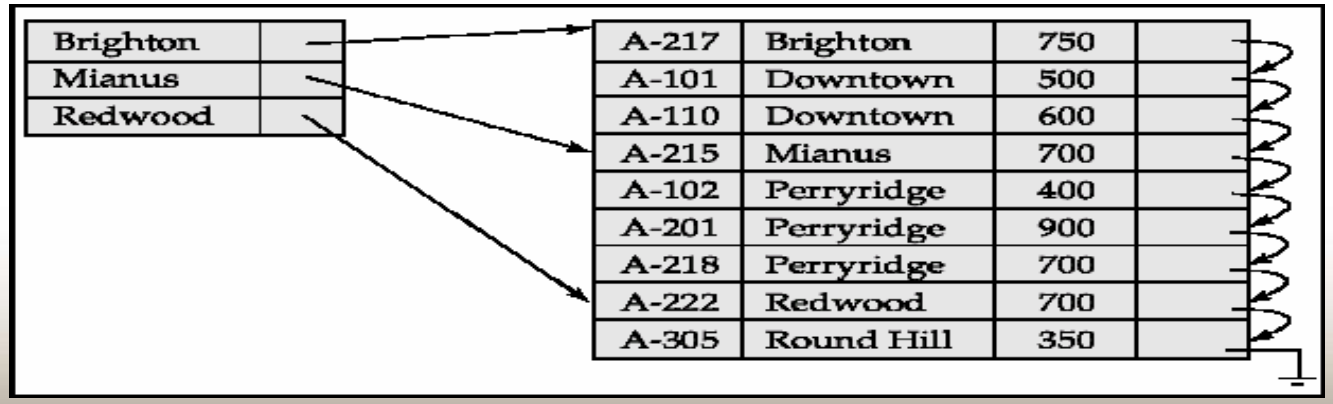
Index file Evaluation Metrics

- Running time:
  - Indexing time;
  - Access/search time;
  - Update time (*Insertion* time, *Deletion* time, *Modification* time….)
- Space overhead:
  - Computer *storage space* consumed.
- Access types supported efficiently.
  - Is the *indexing structure* allows to access:
    - Records with *a specified term*, or
    - Records with *terms falling* in a specified range of values.

fppt.com

Sequential File

- Sequential file is the *most primitive* file structures.
  - It has *no vocabulary* as well as *linking pointers*.
- The records are generally *arranged serially*, one after another, but in *lexicographic order* on the value of some key field.
  - A *particular attribute* is chosen as *primary key* whose value will determine the *order of the records*.
  - When the *first key fails* to discriminate among records, a *second key* is chosen to give an order.



fppt.com

Example:

- Given a *collection of documents*, they are parsed to extract words and these are saved with the Document ID.

Doc 1

I did enact Julius
Caesar I was killed
I the Capitol;
Brutus killed me.

Doc 2

So let it be with
Caesar. The noble
Brutus has told you
Caesar was ambitious

fppt.com

Sorting the Vocabulary

- After all documents have been tokenized, stopwords are removed, and normalization and stemming are applied, to generate index terms

- These index terms in sequential file are *sorted in alphabetical order.*

Term	Doc #
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
I	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Sequential file

	Term	Doc No.
1	ambition	2
2	brutus	1
3	brutus	2
4	capitol	1
5	caesar	1
6	caesar	2
7	caesar	2
8	enact	1
9	julius	1
10	kill	1
11	kill	1
12	noble	2

fppt.com

Sequential File

- Its *main advantages* are:
  - *Easy* to implement;
  - Provides fast access to the next record using *lexicographic order*.
  - Instead of *Linear time search*, one can search in *logarithmic time* using *binary search*.
- Its disadvantages:
  - Difficult to *update*. Index must be *rebuilt if a new term is added*. Inserting a new record may *require moving a large proportion* of the file;
  - Random access is *extremely slow*.
- The problem of update *can be solved* :
  - By ordering records by *date of acquisition*, than the key value; hence, the newest entries are *added at the end of the file* & therefore *pose no difficulty to updating*.
  - But searching becomes very tough; it requires *linear time*.

fppt.com

Inverted file

- A *word oriented indexing mechanism* based on sorted list of keywords, with each keyword having links to *the documents containing it*.
  - Building and maintaining *an inverted index* is a relatively low cost risk.
  - On a text of *n words* an inverted index can be built in O(n) time, n is number of keywords.

- Content of the *inverted file*:
  - Data to be held in the inverted file includes :
    - The *vocabulary* (List of terms)
    - The *occurrence* (Location and frequency of terms in a document collection)

fppt.com

Inverted file

- The *occurrence*: contains *one record per term*, listing.
  - Frequency of each term in a document, i.e. *count number* of occurrences of keywords in a document.
    - TF_{ij}, *number of occurrences* of term t_j in document $\mathbf{d}_i$
    - DF_j, *number of documents* containing t_j
    - max_i, *maximum frequency* of any term in $\mathbf{d}_i$
    - N, *total number of documents* in a collection
    - CF_j,, *collection frequency* of t_j in n_j
    - ….
  - *Locations/Positions* of words in the text.

fppt.com

Inverted file

- Why vocabulary?
  - Having information about vocabulary (list of terms) *speeds searching for relevant documents*.
- Why location?
  - Having information about the location of each term within the document helps for:
    - User interface design: highlight location of search term,
    - Proximity based ranking: adjacency and near operators (in Boolean searching)
- Why frequencies?
  - Having *information about frequency* is used for:
    - Calculating *term weighting* (like TF, TF*IDF, …)
    - Optimizing query processing.

fppt.com

Inverted File

- Documents are organized by the *terms/words* they contain

Term	***CF***	***Document ID***	***TF***	***Location***
auto	3	2 19 29	1 1 1	66 213 45
bus	4	3 19 22	1 2 1	94 7, 212 56
taxi	1	5	1	43
train	3	11 34	2 1	3, 70 40

This is called an ***index file***.

Text operations are performed before *building the index*.

fppt.com

Construction of Inverted File

- An inverted index consists of *two files*:
  - Vocabulary file
  - Posting file

Advantage of dividing *inverted file*:

- Keeping a *pointer in the vocabulary* to the list in the posting file allows:
  - The vocabulary to be *kept in memory* at search time even for *large text collection*, and
  - Posting file to be kept *on disk for accessing* to documents.

fppt.com

Inverted Index Storage

- Separation of inverted file into *vocabulary* and *posting file* is a good idea.
  - *Vocabulary*: For searching purpose we need only *word list*.
  - This allows the vocabulary to be *kept in memory* at search time since the space required for *the vocabulary is small*.
    - The vocabulary grows by $O(n^{\beta})$, where β is a *constant between 0 – 1*.
    - Example: from 1,000,000,000 documents, there may be 1,000,000 distinct words. Hence, the size of index is 100 MBs, which can easily be *held in memory of a dedicated computer*.
  - *Posting file* requires *much more space*.
    - For each word appearing in the text we are *keeping statistical* information related to word occurrence in documents.
    - Each of the *postings pointer to the document requires* an extra space of O(n).
- How to speed up *access to inverted file*?

fppt.com

Vocabulary File

- A vocabulary file (Word list):
  - Stores all of the *distinct terms* (*keywords*) that appear in any of the documents (in lexicographical order) and
  - For each word a *pointer to posting file*.

- Records kept for each term j in the *word list contains* the following:
  - Term j
  - Number of documents in which term j occurs (DF_j)
  - Total frequency of term j (CF_j)
  - Pointer to postings (inverted) list for term j

fppt.com

Postings File (Inverted List)

- For each distinct term in the vocabulary, stores a *list of pointers to the documents* that contain that term.
- Each element in an *inverted list is called a posting*, i.e., the occurrence of a term in a document.
- It is stored as a *separate inverted list for each column*, i.e., a list corresponding to each term in the index file.
  - Each list consists of *one or many individual postings related to Document ID*, TF and location information about a given term i.

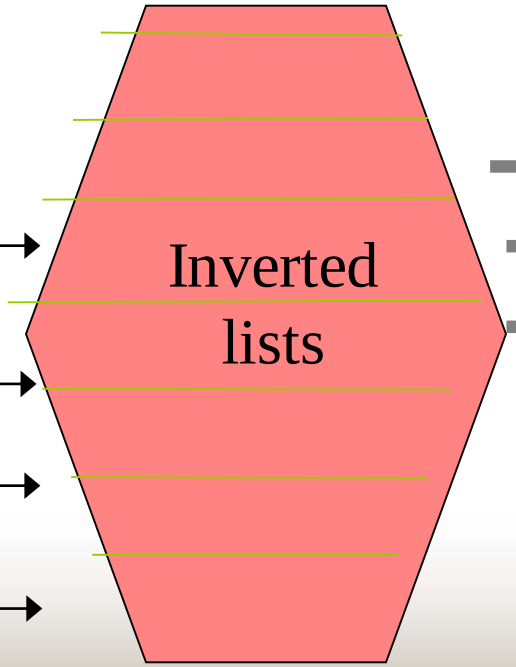
fppt.com

Organization of Index File

Vocabulary
(word list)

Term	No of Doc	Tot freq	Pointer To posting
Act	3	3	
Bus	3	4	
pen	1	1	
total	2	3	

Postings
(inverted list)



Actual
Documents

fppt.com

Example: Indexing

- Given a *collection of documents*, they are parsed to extract words and these are saved with the Document ID.

Doc 1

I did enact Julius
Caesar I was killed
I the Capitol;
Brutus killed me.

Doc 2

So let it be with
Caesar. The noble
Brutus has told you
Caesar was ambitious

fppt.com

Sorting the Vocabulary

- After all documents have been tokenized the inverted file is *sorted by terms*.

Term	Doc #
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
I	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Term	Doc #
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
has	1
I	1
I	1
I	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

fppt.com

Remove stopwords, Apply Stemming & Compute Term Frequency

- Multiple term entries in a single document are *merged and frequency information added*.
- Counting number of occurrence of terms in the *collections helps to compute TF*.

Term	Doc #
ambition	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
enact	1
julius	1
kill	1
kill	1
noble	2

Term	Doc #	TF
ambition	2	1
brutus	1	1
brutus	2	1
capitol	1	1
caesar	1	1
caesar	2	2
enact	1	1
julius	1	1
kill	1	2
noble	2	1

fppt.com

Vocabulary and Postings File

❖ The file is commonly *split into a Dictionary and a Posting file*.

Term	Doc #	TF
ambition	2	1
brutus	1	1
brutus	2	1
capitol	1	1
caesar	1	1
caesar	2	2
enact	1	1
julius	1	1
kill	1	2
noble	2	1

Vocabulary

Term	DF	CF
ambitious	1	1
brutus	2	2
capitol	1	1
caesar	2	3
enact	1	1
julius	1	1
kill	1	2
noble	1	1

Posting

Doc #	TF
2	1
1	1
2	1
1	1
1	1
2	2
1	1
1	1
1	2
2	1

Pointers

fppt.com

Complexity Analysis

- The *inverted index* can be built in O(n) time.
  - n is number of vocabulary terms.
- Since terms in *vocabulary file* are sorted searching takes *logarithmic time*.
- To update the inverted index it is possible to apply *Incremental indexing* which requires O(k) time, k is number of new index terms.

fppt.com

Exercises

- Construct the *inverted index* for the following document collections.
  - *Doc 1* : New home to home sales forecasts
  - *Doc 2* : Rise in home sales in July
  - *Doc 3* : Home sales rise in July for new homes
  - *Doc 4* : July new home sales rise

fppt.com

Implementation Issues

- Storage of text:
  - The need for *text compression*: to reduce storage space.
- Indexing text
  - Storage of indexes
    - Is compression required? Do we store on *memory or in a disk* ?
- Accessing text
  - Accessing indexes
    - How to *access to indexes*? What *data/file structure* to use?
  - Processing indexes
    - How to a *search a given query in the index*? How to *update the index*?
  - Accessing documents

fppt.com

Text Compression

- Text compression is about finding ways to *represent the text in fewer bits or bytes*. Advantages:
  - Save *storage space* requirement.
  - *Speed up* document transmission time
  - Takes *less time to search* the compressed text
- Common compression methods:
  - ***Statistical methods***: which requires *statistical information* about frequency of occurrence of symbols in the document.
  - E.g. Huffman coding
    - Estimate *probabilities of symbols*, code one at a time, *shorter codes* for high probabilities.
  - ***Adaptive methods***: which constructs dictionary in the processing of compression.
  - E.g. Ziv-Lempel compression:
    - Replace *words or symbols with a pointer* to dictionary entries.

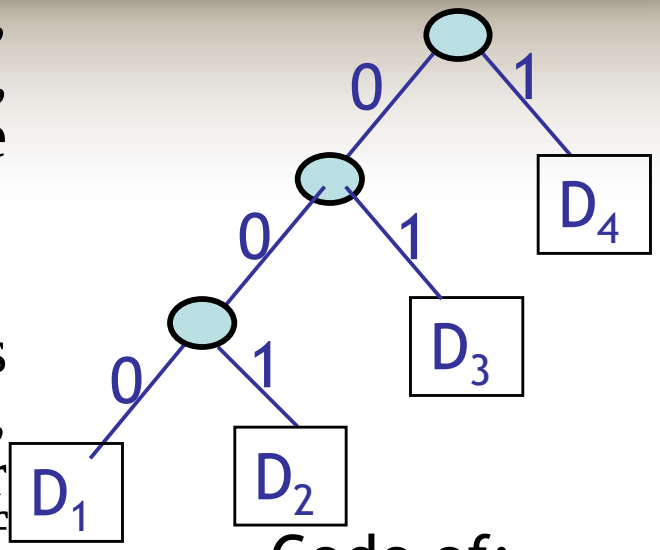
fppt.com

Huffman Coding

- Developed in 1950s by David Huffman, widely used *for text compression*, *multimedia codec* and message transmission.

- ***The problem***: Given a set of n symbols and their weights (or frequencies), construct a tree structure (a binary tree for binary code) with the objective of *reducing memory space & decoding* time per symbol.

- Huffman coding is constructed based on *frequency of occurrence* of letters in text documents.



Code of:

D_1 = 000

D_2 = 001

D_3 = 01

D_4 = 1

fppt.com

How to Construct Huffman Coding

- Step 1: Create forest of trees for *each symbol*, $t_1, t_2, \ldots t_n$
- Step 2: Sort forest of trees according to *falling probabilities* of symbol occurrence.
- Step 3: WHILE more than one tree exist DO.
  - Merge two trees t_1 and t_2 with least probabilities p_1 and p_2.
  - Label their root with sum $p_1 + p_2$
  - Associate binary code: 1 with the *right branch* and 0 with the *left branch*.
- Step 4: Create a unique codeword *for each symbol by traversing* the tree from the root to the leaf.
  - Concatenate all encountered 0s and 1s together during traversal.
- The resulting tree has a *prob. of 1 in its root* and *symbols* in its leaf node.

fppt.com

Example

- Consider a *7-symbol alphabet* given in the following table to construct the Huffman coding.

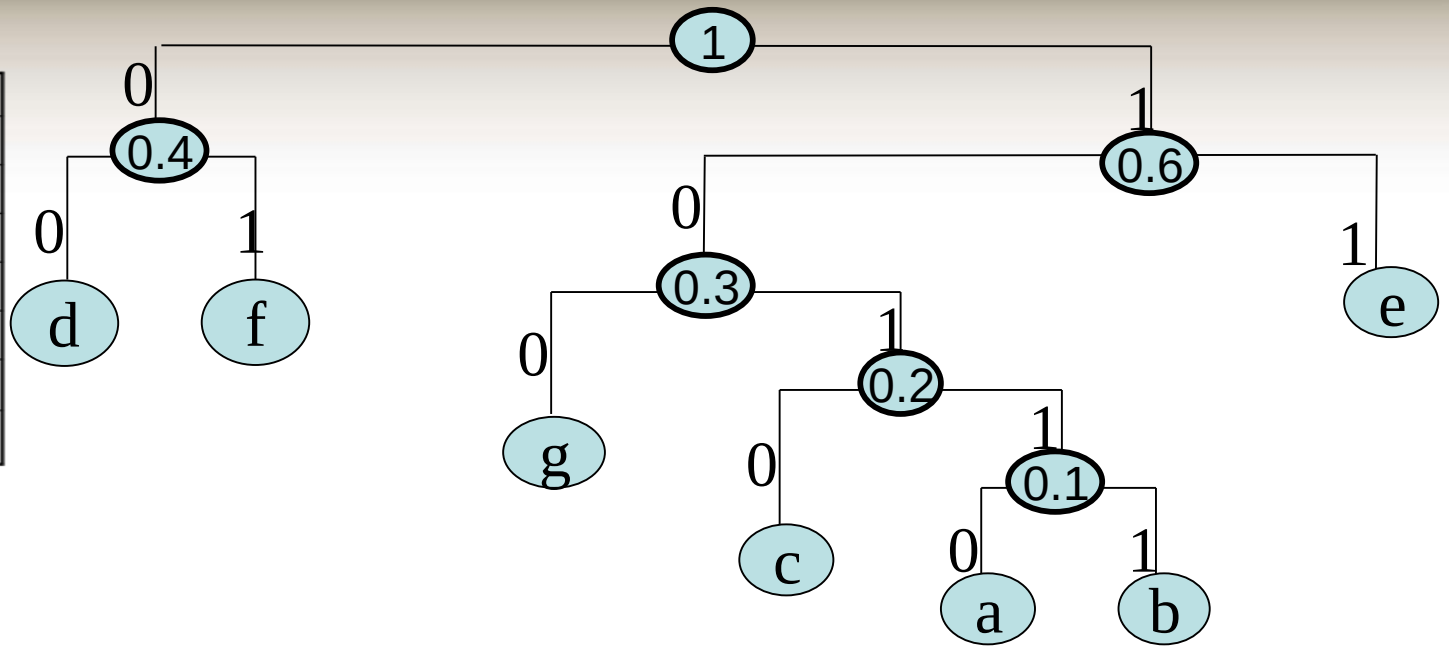
Symbol	***Probability***
a	0.05
b	0.05
c	0.1
d	0.2
e	0.3
f	0.2
g	0.1

- The Huffman encoding algorithm picks each time two symbols (with the smallest frequency) to combine.

fppt.com

Huffman code tree

Symbol	Probability
a	0.05
b	0.05
c	0.1
d	0.2
e	0.3
f	0.2
g	0.1



- Using the Huffman coding a table can be constructed by *working down the tree, left to right*. This gives the binary equivalents for each symbol in terms of *1s and 0s*.
- What is the Huffman binary representation for '*café*'?

fppt.com

Exercise

- 1. Given the following, apply the *Huffman algorithm* to find an *optimal binary code*:

Character:	a	b	c	d	e	t
Frequency:	16	5	12	17	10	25

- 2. *Given* text: “for each rose, a rose is a rose”
  - Construct the Huffman coding

fppt.com

Ziv-Lempel Compression

- The problem with Huffman coding is that it *requires knowledge* about the data before encoding takes place.
  - Huffman coding requires frequencies of *symbol occurrence* before codeword is assigned to symbols.
- Ziv-Lempel compression
  - Not rely on *previous knowledge* about the data.
  - Rather builds this knowledge in the course of *data transmission/data storage*.
  - Ziv-Lempel algorithm (called LZ) uses a table of code-words created *during data transmission*;
    - Each time it replaces strings of characters with a reference to a previous occurrence of the string.

fppt.com

Lempel-Ziv Compression Algorithm

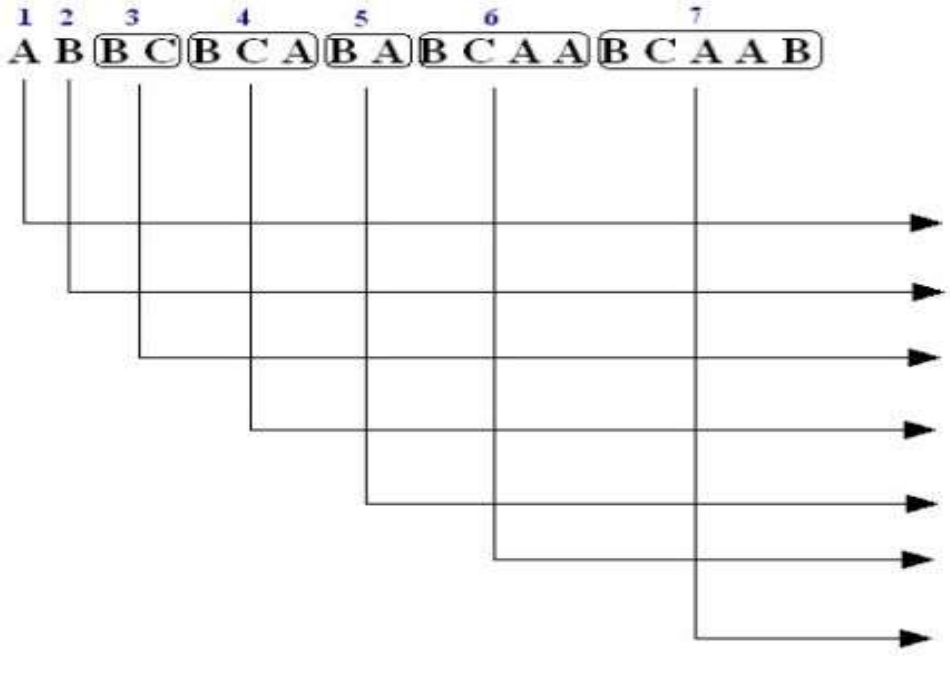
- The multi-symbol patterns are of the form: $C_0C_1 \ldots C_{n-1}C_n$. The prefix of a pattern consists of all the pattern symbols except the last: $C_0C_1 \ldots C_{n-1}$

- Lempel-Ziv Output: there are *three options in assigning a code* to each symbol in the list.
  - If one-symbol pattern is *not in dictionary*, assign (0, symbol)
  - If *multi-symbol pattern is not* in dictionary, assign (dictionaryPrefixIndex, lastPatternSymbol)
  - If the *last input symbol or the last pattern* is in the dictionary, assign (dictionaryPrefixIndex,)

fppt.com

Example: LZ Compression

- Encode (i.e., compress) the string ABBCBCABABCAABCAAB using the LZ algorithm.



	Dictionary	
output	index	string
(0, A)	1	A
(0, B)	2	B
(2, C)	3	BC
(3, A)	4	BCA
(2, A)	5	BA
(4, A)	6	BCAA
(6, B)	7	BCAAB

- The compressed message is: 0A0B2C3A2A4A6B

fppt.com

Example: Decompression

❖ Decode (i.e., decompress) the sequence: 0A0B2C3A2A4A6B

1	2	3	4	5	6	7
(0, A)	(0, B)	(2, C)	(3, A)	(2, A)	(4, A)	(6, B)
A	B	string(2) + C	string(3) + A	string(2) + A	string(4) + A	string(6) + B

	Dictionary	
output	index	string
A	1	A
B	2	B
BC	3	BC
BCA	4	BCA
BA	5	BA
BCAA	6	BCAA
BCAAB	7	BCAAB

The decompressed message is:
ABBCBCABABCAABCAAB

fppt.com

Exercise

- Encode (i.e., compress) the following strings using the *Lempel-Ziv algorithm*.

  1. Mississippi
  2. ABBCBCABABCAABCAAB
  3. SATATASACITASA.

fppt.com

Indexing Structures: Assignment

- Discuss in detail *theoretical and algorithmic concepts* (including construction, various operations, complexity, etc.) on the following commonly used data structures:
  1. *Data structure vs. file structure*
  2. *Arrays (fixed and dynamic arrays), sorted arrays*
  3. *Records and linked list*
  4. *Tree (AVL tree, Binary tree): balanced vs. unbalanced tree)*
  5. *B tree and its variants (B+, B++ Tree, B* Tree),*
  6. *Hierarchical Tree (like Quad Tree and its variants)*
  7. *PAT-Tree and its variants*
  8. *Disjoint tree: balanced and degenerate tree*
  9. *Graph*
  10. *. Hashing,*
  11. *Trie and its variants*

fppt.com

Question & Answer

fppt.com

Thank You !!!

fppt.com